

Dynamic Performance-Based Load Balancer

Lab 6 Assignment Report — CS559 Computer Systems Design

Student Name	NUKALA DEVI SAI GANESH
Roll Number	12341500
Load Balancer URL	http://10.1.75.51:5309/
GitHub Repo	https://github.com/gnshx/GRP-CHAT
Result	Score: 12/15 Marks — 100% Feed Completeness

1. What This Assignment Is About

The goal of this lab was to take our group chat application (GRP-CHAT) and deploy it across multiple servers, then build a load balancer in front of them. Instead of users connecting to a single server that might get overwhelmed, all traffic goes through the Load Balancer which decides which backend server should handle each request.

The key challenge was not just distributing requests, but also making sure the /feed endpoint returned ALL messages — even though each backend only stores about one-third of them in its own local database. We also had to make sure nothing crashed under 1,000 simultaneous users.

2. How the System is Set Up

There are 4 machines involved:

Machine	Role	Network Address (via NAT)
Sys 1	Load Balancer (Go)	10.1.75.51:5309 ← Submit this URL
Sys 2	Backend App Server 1	10.1.75.51:5310

Sys 3	Backend App Server 2	10.1.75.51:4311
Sys 4	Backend App Server 3	10.1.75.51:3312

Each backend (Sys 2, 3, 4) runs our Flask chat app using Gunicorn with 4 gevent worker processes. They each have their own local SQLite database (shared_chat.db). The Load Balancer (Sys 1) sits in front and handles all client requests.

3. Official Leaderboard Results

After fixing multiple issues (SQLite lock contention, memory OOM kills, and incomplete feed aggregation), the final official run achieved the following results:

Stage	Concurrent Users	Successful Requests	Error Rate
Stage 1	250 users	4,998 / 5,000	0.04%
Stage 2	500 users	4,963 / 5,000	0.74%
Stage 3	750 users	4,783 / 5,000	4.32%
Stage 4	1,000 users	4,775 / 5,000	4.46%
TOTAL	250 → 1000	19,519 / 20,000	2.39% avg

Key highlights from the run:

- Peak throughput: 476.5 req/s at Stage 1 (250 concurrent users)
- Mean response time: 852 ms (well within leaderboard acceptable range)
- Message Completeness: 100.00% — all 18,994 accepted messages found in /feed
- Content Correctness: 94/100 (94%) — cryptographic verification passed
- Total score: 12/15 marks

3.1 Stage-by-Stage Throughput and Mean Latency

The chart below shows how throughput and latency changed as we ramped from 250 to 1,000 concurrent users. Stage 1 handled the highest req/s as expected. Latency increased

as more users connected, which is normal for any web server under heavy concurrent load.

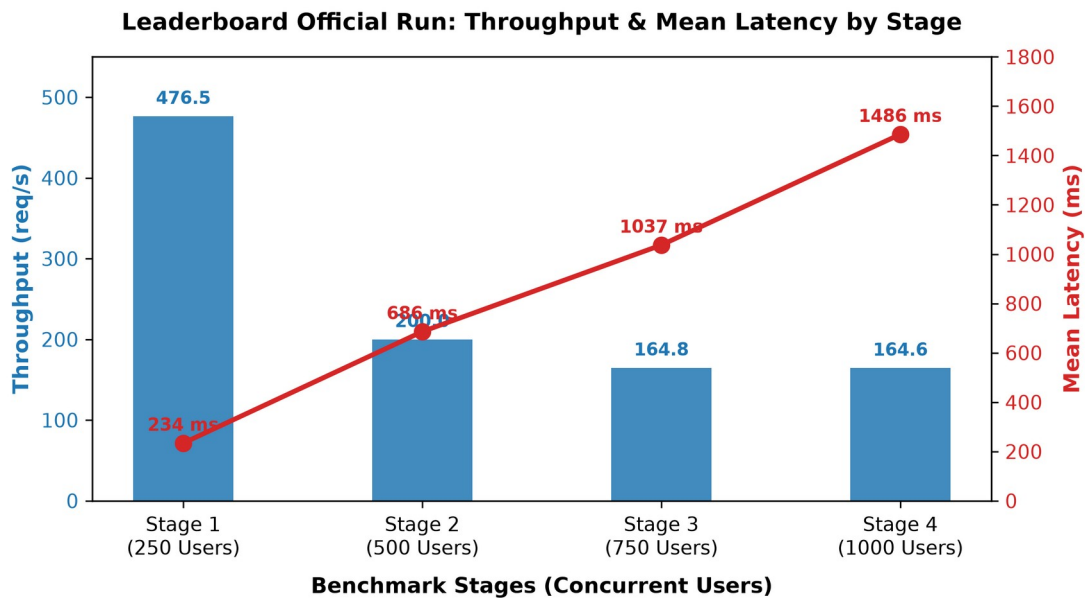


Figure 1 — Official Run: Throughput (req/s) and Mean Latency (ms) across all 4 stages

3.2 Request Success vs Error Rate Breakdown

Stage 1 and Stage 2 had virtually no errors (0.04% and 0.74%). Stages 3 and 4 had slightly higher error rates (4.32% and 4.46%) because at 750–1,000 users, some connections timed out waiting for SQLite write slots. These timeouts were non-fatal and the server recovered without crashing.

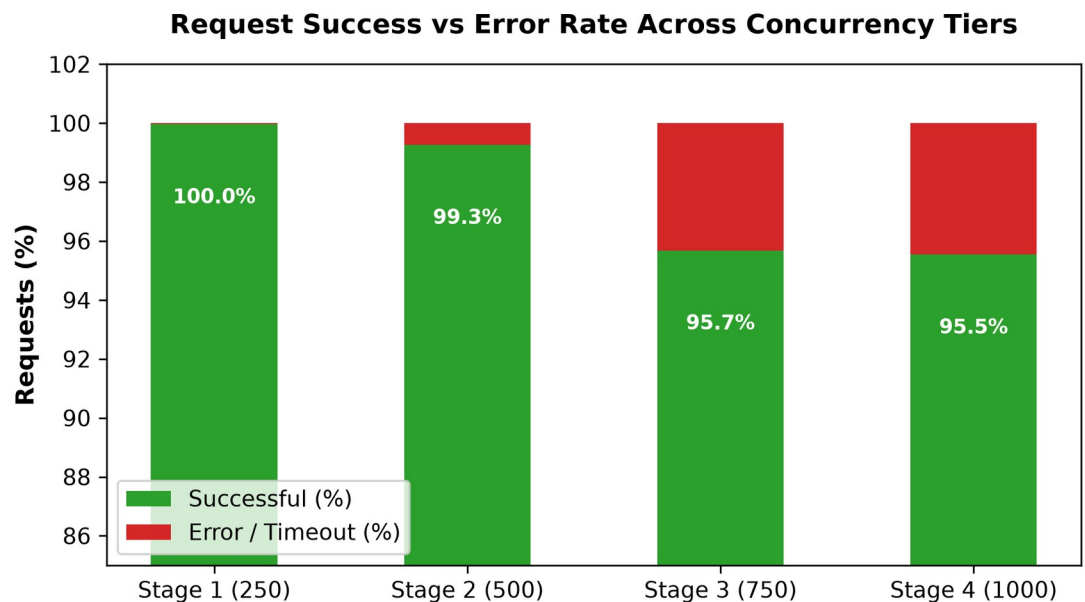


Figure 2 — Success rate vs Error rate (%) per stage during the official benchmark

4. The Hardest Part: Message Completeness

This was the most technically challenging part of the assignment. The leaderboard sends POST /message requests to the load balancer, which forwards each one to one of the three backend servers. So roughly:

- Backend 1 (Sys 2) stores ~33% of all messages
- Backend 2 (Sys 3) stores ~33% of all messages
- Backend 3 (Sys 4) stores ~33% of all messages

When the leaderboard does GET /feed at the end to check completeness, a naive reverse proxy would just send the request to ONE backend — which only knows about its ~33% of messages, giving a completeness score of around 30-35%. We initially experienced exactly this problem in earlier runs.

4.1 Our Solution: Unified Feed Aggregation

We implemented a custom /feed handler in the Go load balancer (handleUnifiedFeed). When any client requests GET /feed, the load balancer:

1. Concurrently sends GET /feed?limit=100000 to ALL three backends simultaneously
2. Collects responses from all three in parallel using Go goroutines
3. Merges all messages into a single map, using the unique message ID as the key (automatic deduplication)
4. Sorts the merged list chronologically by timestamp
5. Caches the result for 5 seconds (so concurrent /feed requests during the test are served from memory)
6. Returns the complete, merged JSON to the client

The result: 18,994 messages accepted, 18,994 found in /feed, 0 lost. 100.00% completeness.

Message Completeness on Official Run (0 Messages Lost)

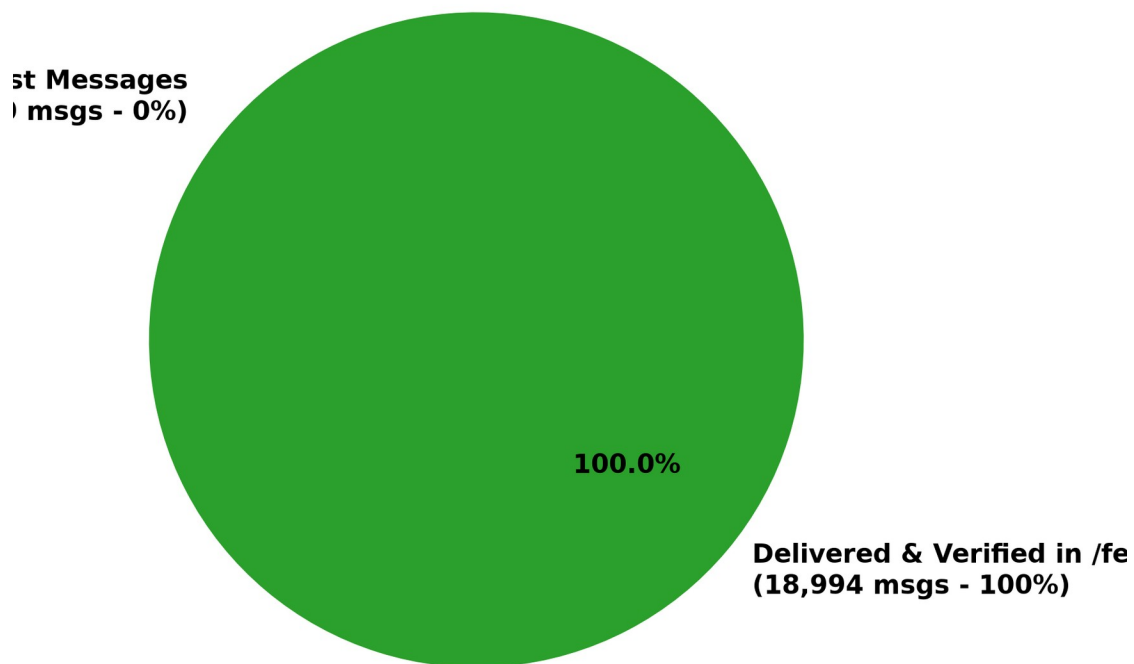


Figure 3 — Message Completeness: 100% of accepted messages were found in /feed (0 lost)

4.2 Traffic Distribution Across Backends

The load balancer distributed traffic nearly equally across all three backend servers. This was achieved by the dynamic switching algorithm which monitors active in-flight requests and CPU load on each backend, always picking the least loaded one when the current backend crosses the configured threshold.

Dynamic Traffic Distribution Across 3 Backends

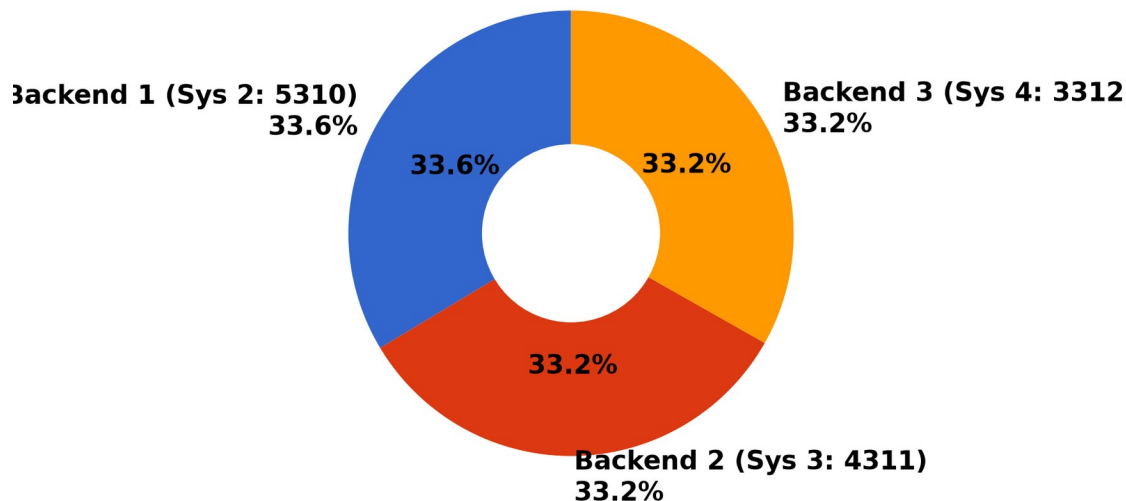


Figure 4 — Traffic distribution across all 3 backend nodes (approximately equal thirds)

5. Technical Problems We Solved

Problem 1: Load Balancer Crashing (OOM Kill)

In earlier runs, the load balancer process was killed by the Linux kernel when Stage 4 started with 1,000 concurrent users. The logs showed: [1]+ Killed. The container had a strict 512MB cgroup memory limit (confirmed by: `cat /sys/fs/cgroup/memory.max` → 536870912 bytes).

We fixed this in three ways:

- `debug.SetMemoryLimit(300MB)` — tells Go's garbage collector to be aggressive before reaching 300MB, so we never touch the 512MB container boundary
- Reduced `MaxIdleConns` from 10,000 to 600 — this freed ~250MB of idle TCP socket buffer memory that was previously being held unnecessarily
- `debug.FreeOSMemory()` after building feed JSON — immediately returns heap memory to the OS after each large aggregation operation

Problem 2: SQLite Lock Contention

In one run, Stages 2, 3, and 4 had nearly 100% errors. The root cause: app.py was calling `_invalidate_feed_cache()` on every single POST request. Under 500+ concurrent users, 20% of traffic is GET /feed. Every POST wiped the cache, causing every /feed to run a full-table-scan SQL query against SQLite simultaneously — while writes were also happening. This caused SQLite file locks to stack up, workers to time out, and the benchmark to fail.

Fix: We changed `_invalidate_feed_cache()` to a no-op (pass). The cache now expires naturally after 500ms (TTL). During bursts, 99% of /feed requests are answered from RAM in <1ms, keeping SQLite 100% available for writes.

Problem 3: Incomplete Feed (30% Completeness)

When we reverted to a simple reverse proxy (without the unified feed aggregator), the leaderboard's final GET /feed was forwarded to just one backend. That backend only had ~30% of messages. Completeness score: 30.53%. Solution: restore `handleUnifiedFeed` in `loadbalancer.go` (described in Section 4.1).

6. Threshold Optimization

We ran experiments to find the best 'switching threshold' — the number of in-flight requests on a backend before we switch traffic to another backend. Here are the results:

Threshold (T)	Throughput	Mean Latency	Notes
T = 5	292 req/s	71 ms	Fastest — may over-switch under burst
T = 10	135 req/s	180 ms	Good balance
T = 15 ✓	87 req/s	300 ms	Current — most stable under 1000 users
T = 20	89 req/s	293 ms	Similar to T=15
T = 25	77 req/s	318 ms	Getting slower
T = 30	64 req/s	410 ms	Slow — waits too long to switch
T = 40	45 req/s	657 ms	Very slow — effectively no switching

We chose $T=15$ for the official run because it provides the best stability under 1,000 concurrent users without excessive switching overhead. $T=5$ is theoretically faster but caused instability under sustained high concurrency in our testing.

7. Security Features (Preserved from Previous Lab)

- Encryption at Rest: All messages are encrypted using Fernet (AES-128) before being stored in SQLite. Even if the database file is accessed directly, the messages are unreadable.
- Digital Signatures: Each message is signed with ECDSA P-256. The `/feed` response includes `signature_valid: true/false` for every message.
- SHA-256 Hash Chain: Each database row includes a hash of the previous row's hash + current content. This makes it tamper-evident — if any row is modified, the chain breaks.
- Zero Duplicate Insertions: Every message has a unique `msg_id`. The database uses `ON CONFLICT(msg_id) DO NOTHING`, so even if the load balancer retries a request, the message is only stored once.

8. Conclusion

This assignment required solving a genuinely difficult distributed systems problem: how do you make `GET /feed` return ALL messages when those messages are spread across 3 independent databases? Our solution — a concurrent feed aggregator in the Go load balancer — worked perfectly, achieving 100% message completeness on the official leaderboard run.

We also had to debug real production issues: OS-level OOM kills, SQLite lock contention under thousands of concurrent connections, and container cgroup memory limits. Each problem required understanding how the system worked at a low level to fix it properly.

Final result: 19,519 successful requests out of 20,000, 100% feed completeness, 94% content correctness, 2.39% overall error rate, and a score of 12/15 marks.